

Lambda Calculus com Tipos

Maria João Frade

HASLab - INESC TEC
Departamento de Informática, Universidade do Minho

2025/2026

Lambda calculus com tipos simples - $\lambda_{\rightarrow}$

- O propósito dos sistemas de tipos é a classificação dos termos.
- A relação entre objectos e tipos é capturada por juízos da forma $e : \tau$, que se lê “ e tem tipo τ ”.
- Os sistemas de tipos foram introduzidos por Bertrand Russell na década de 1900.
- Em 1940 A. Church introduziu o *lambda calculus com tipos simples*.
- Uma versão diferente do lambda calculus tipificado tinha já sido apresentada por H. Curry em 1934, para a lógica combinatorial.
- Na versão de Church os termos têm anotações de tipo, enquanto que na versão de Curry os termos têm a mesma sintaxe abstracta que no lambda calculus puro.

Sintaxe

Tipos

- Seja $\mathcal{G}$ um conjunto não vazio de *tipos de base*.
- Os tipos são definidos pela seguinte sintaxe abstracta

$$\tau, \sigma ::= T \mid \tau \rightarrow \sigma \quad \text{onde } T \in \mathcal{G}$$

Termos

- Assume-se um conjunto enumerável de *variáveis*: $x, y, z, \dots$
- Fixamos um conjunto das *constantes* dos diferentes tipos
- Os termos são definidos pela seguinte sintaxe abstracta

► Versão de Church

$$e, a, b ::= c \mid x \mid \lambda x:\tau. e \mid a b \quad \text{onde } c \text{ é uma constante}$$

► Versão de Curry

$$e, a, b ::= c \mid x \mid \lambda x. e \mid a b \quad \text{onde } c \text{ é uma constante}$$

Variáveis livres e ligadas

$FV(e)$ denota o conjunto das *variáveis livres* de uma expressão e

$$\begin{aligned} FV(c) &= \{\} \\ FV(x) &= \{x\} \\ FV(\lambda x:\tau. a) &= FV(a) \setminus \{x\} \\ FV(a b) &= FV(a) \cup FV(b) \end{aligned}$$

$BV(e)$ denota o conjunto das *variáveis ligadas* de uma expressão e

$$\begin{aligned} BV(c) &= \{\} \\ BV(x) &= \{\} \\ BV(\lambda x:\tau. a) &= BV(a) \cup \{x\} \\ BV(a b) &= BV(a) \cup BV(b) \end{aligned}$$

Uma variável pode ser simultaneamente livre e ligada numa dada expressão.
Por exemplo,

$$(x y) (\lambda z:\tau. \lambda x:\tau \rightarrow \sigma. x z)$$

Convenções

Para evitar parentesis segue-se a seguinte convenção:

- a construção de tipos $\rightarrow$ é associativa à direita;
- a aplicação é associativa à esquerda;
- o âmbito da abstração λ estende-se para a direita o mais possível.

α -conversão

$$\lambda x:\tau. e = \lambda y:\tau. e[y/x] \quad , \text{ se } y \notin \text{FV}(e)$$

Esta conversão induz uma relação de equivalência nos termos.

Convenção das variáveis

- Identificamos os termos que são iguais a menos de renomeação de variáveis ligadas (α -conversão). Exemplo: $(\lambda x:\tau. y x) = (\lambda z:\tau. y z)$.
- Todas as variáveis ligadas são escolhidas de forma a serem diferentes das variáveis livres.

Substituição

A conversão das variáveis permite definir a substituição do seguinte modo:

Substituição

$$\begin{aligned} c[a/x] &= c \\ x[a/x] &= a \\ y[a/x] &= y && \text{se } x \neq y \\ (\lambda y:\tau. b)[a/x] &= (\lambda y:\tau. b[a/x]) \\ (e_1 e_2)[a/x] &= (e_1[a/x]) (e_2[a/x]) \end{aligned}$$

Lema da substituição

Sejam x e y variáveis distintas e $x \notin \text{FV}(e)$, então

$$(a[b/x])[e/y] = (a[e/y])[b[e/y]/x]$$

β -redução

A redução β indica o efeito de aplicar uma função a um argumento.

β -redução

A β -redução, $\rightarrow_\beta$, é definida como o fecho compatível da regra

$$(\lambda x:\tau. a) b \rightarrow_\beta a[b/x]$$

- um termo $(\lambda x:\tau. a) b$ chama-se β -redex e a $a[b/x]$ o seu *contractum*
- $\rightarrow_\beta^*$ é fecho reflexivo e transitivo de $\rightarrow_\beta$
- $=_\beta$ é fecho reflexivo, simétrico e transitivo de $\rightarrow_\beta$

Uma expressão que não contém nenhum β -redex diz-se uma *forma normal*.

Uma expressão e diz-se *fortemente normalizável* se todas as sequências de redução com origem em e terminam.

Sistema de tipos

No lambda calculus com tipos as funções:

- são classificadas com tipos simples que determinam o tipo dos seus argumentos e o tipo dos valores que elas produzem;
- só podem ser aplicadas a argumentos de tipo apropriado.

Para definir a relação de tipificação entre termos e tipos precisamos de introduzir o conceito de *contexto*, para declarar os tipos das variáveis livres.

Sistema de tipos

Contexto

- Um *contexto* Γ é um conjunto finito de declarações de variáveis $\{x_1 : \tau_1, \dots, x_n : \tau_n\}$ onde as variáveis x_i são todas distintas.
- Γ pode ser visto como uma função parcial: $\text{dom}(\Gamma) = \{x_1, \dots, x_n\}$ e $\Gamma(x_i) = \tau_i$.
- Usualmente escrevemos $x_1 : \tau_1, \dots, x_n : \tau_n$ em vez de $\{x_1 : \tau_1, \dots, x_n : \tau_n\}$, e quando escrevemos $\Gamma, x : \tau$ ou Γ, Γ' assumimos implicitamente que $x \notin \text{dom}(\Gamma)$ e $\text{dom}(\Gamma) \cap \text{dom}(\Gamma') = \emptyset$.

Um juízo de tipificação é um triplo da forma

$$\Gamma \vdash e : \sigma$$

onde Γ é um contexto, e um termo e σ um tipo.

Sistema de tipos

Regras de inferência de tipos

(var)	$\overline{\Gamma \vdash x : \sigma}$	se $(x : \sigma) \in \Gamma$
(const)	$\overline{\Gamma \vdash c : T}$	se c tem tipo T
(abs)	$\frac{\Gamma, x : \tau \vdash e : \sigma}{\Gamma \vdash (\lambda x : \tau. e) : \tau \rightarrow \sigma}$	
(app)	$\frac{\Gamma \vdash a : \tau \rightarrow \sigma \quad \Gamma \vdash b : \tau}{\Gamma \vdash (a \ b) : \sigma}$	

Um termo e diz-se *bem tipificado* se $\Gamma \vdash e : \sigma$ para algum Γ e σ .

Sistema de tipos

Árvore de tipificação de $z : \tau \vdash (\lambda y : \tau \rightarrow \tau. yz)(\lambda x : \tau. x) : \tau$

$$\frac{\frac{\frac{\frac{\frac{\overline{z : \tau, y : \tau \rightarrow \tau \vdash y : \tau \rightarrow \tau} \text{ (var)}}{z : \tau, y : \tau \rightarrow \tau \vdash yz : \tau} \text{ (app)}}{z : \tau \vdash (\lambda y : \tau \rightarrow \tau. yz) : (\tau \rightarrow \tau) \rightarrow \tau} \text{ (abs)}}{z : \tau \vdash (\lambda y : \tau \rightarrow \tau. yz)(\lambda x : \tau. x) : \tau} \text{ (app)} \quad \frac{\frac{\frac{\overline{z : \tau, x : \tau \rightarrow \tau \vdash z : \tau} \text{ (var)}}{z : \tau, x : \tau \rightarrow \tau \vdash x : \tau} \text{ (app)}}{z : \tau \vdash (\lambda x : \tau. x) : \tau \rightarrow \tau} \text{ (abs)}}{z : \tau \vdash (\lambda y : \tau \rightarrow \tau. yz)(\lambda x : \tau. x) : \tau} \text{ (app)}$$

A mesma árvore apresentada de forma tabular:

1. $z : \tau \vdash (\lambda y : \tau \rightarrow \tau. yz)(\lambda x : \tau. x) : \tau$ (app)
 - 1.1. $z : \tau \vdash (\lambda y : \tau \rightarrow \tau. yz) : (\tau \rightarrow \tau) \rightarrow \tau$ (abs)
 - 1.1.1. $z : \tau, y : \tau \rightarrow \tau \vdash yz : \tau$ (app)
 - 1.1.1.1. $z : \tau, y : \tau \rightarrow \tau \vdash y : \tau \rightarrow \tau$ (var)
 - 1.1.1.2. $z : \tau, x : \tau \rightarrow \tau \vdash z : \tau$ (var)
 - 1.2. $z : \tau \vdash (\lambda x : \tau. x) : \tau \rightarrow \tau$ (abs)
 - 1.2.1. $z : \tau, x : \tau \vdash x : \tau$ (var)

Propriedades

Unicidade de tipos

Se $\Gamma \vdash a : \sigma$ e $\Gamma \vdash a : \tau$, então $\sigma = \tau$.

Inferência de tipos

O problema da inferência de tipos é decidível. Ou seja, é possível deduzir automaticamente o tipo de um termo num dado contexto, caso exista.

Preservação de tipos

Se $\Gamma \vdash a : \sigma$ e $a \rightarrow_{\beta}^* b$, então $\Gamma \vdash b : \sigma$.

Normalização forte

Se $\Gamma \vdash e : \sigma$, então todas as sequências de β -reduções com origem e terminam. Isto é, e é fortemente normalizável.

Propriedades

Confluência

Se $a =_{\beta} b$, então $a \rightarrow_{\beta}^* e$ e $b \rightarrow_{\beta}^* e$, para algum termo e .

Propriedade da substituição

Se $\Gamma, x : \tau \vdash a : \sigma$ e $\Gamma \vdash b : \tau$, então $\Gamma \vdash a[b/x] : \sigma$.

Enfraquecimento

Se $\Gamma \vdash e : \sigma$ e $\Gamma \subseteq \Gamma'$, então $\Gamma' \vdash e : \sigma$.

Fortalecimento

Se $\Gamma, x : \tau \vdash e : \sigma$ e $x \notin FV(e)$, então $\Gamma \vdash e : \sigma$.

Exercícios

Apresente (se possível) juízos de tipificação para os seguintes termos (omitimos aqui as anotações de tipo nos termos para a simplificar a apresentação).

- $\lambda f. \lambda y. f y y$
- $\lambda g. \lambda x. \lambda y. \lambda z. g (x z) (y z)$
- $(\lambda x. x x)$
- $(\lambda f. \lambda y. f y y) (\lambda f. \lambda y. f y y)$
- $(\lambda y. y 1) ((\lambda x. f x 3) 7)$

Exercícios

Escreva as anotações de tipo para os termos

$$\begin{aligned} K &\equiv (\lambda x. \lambda y. x) \\ S &\equiv (\lambda x. \lambda y. \lambda z. x z (y z)) \end{aligned}$$

de forma a que eles sejam termos bem tipificados, e indique os seu tipos. Apresente as árvores de derivação no sistema de tipos que justificam as suas respostas.

Construa a árvore de prova do seguinte juízo de tipificação:

$$h : \text{Int} \rightarrow \text{Int} \rightarrow \text{Int} \vdash (\lambda f : \text{Int} \rightarrow \text{Int}. \lambda x : \text{Int}. f (f x)) (\lambda y : \text{Int}. h y 2)$$